CMPT276 D300 Assignment 4 Report

Nathaniel Chen

March 31, 2026

Course: CMPT276 D300

1 Code Smells Identified

The teammate that I was tasked with reviewing was Maryna De Almeida Brito. I reviewed MovingEnemy.java, Pirate.java, Animal.java and Explorer.java, all of which Maryna had implemented herself during the coding phase. Of those four classes that I reviewed, I was able to identify six smells: three of them were found in **MovingEnemy.java**, two in **Explorer.java**, and one smell spread out between these classes.

MovingEnemy.java

1. **Variable Typo**
   The Position variable called "defaulPosition" has a typo, as it was likely meant to be named "defaultPosition" instead. Although miniscule, this smell does make the code more confusing and causes readability issues with the code.

2. **Changing Variables Before Validating**
   Within the move method, the pixelX and pixelY values are changed and then checked to see if the new pixels are not obstructed by another object. If the new pixel locations are inside of another object, these variables are temporarily in an invalid area before they're reverted. In my opinion, this is poorly structured code, as this method can be refactored to not introduce that invalid state to the pixelX and pixelY variables so that there is no chance that the movingEnemy object will move into an invalid position.

3. **Missing JavaDoc Comments**
   The getVelocictyX and getVelocityY methods do not have JavaDoc comments. Although this does not affect the code much, it is still lack of documentation and therefore a smell.

4. **Missing Encapsulation**
   The defaulPosition, directions and random variables are package private variables, meaning they don't have "public" or "private" before them. Although only other classes within this package treat these variables as public, and classes outside this package treat it as private, it still breaks down encapsulation. This is an unnecessary use of unsafe constructs.

Explorer.java

1. **Incomplete Shield Deactivation Method**
   The deactiveShield method does not set the shieldTicksRemaining variable to 0. This means that, if

other classes were to deactivate the shield when the shieldTicksRemaining value is not 0, the shield can enter a state where it's deactivted but the remaining shield time is still a non-zero number. This can introduce bugs, and is therefore poorly structured code.

2. **Unnecessary Else-If Statement**
   The else-if statement inside of the collectReward method is not necessary, as both the shield and coin class contain a value; the shield class just has a value of 0, so updating the score with the shield value won't have an affect on the score.

3. **Missing Encapsulation**
   The game variable is a package private variable, once again meaning that classes within the Explorer.java's package treat it as a public variable, whereas other classes outside the package treat it as private.This is once again an unnecessary use of unsafe constructs.

2 Smells Identified by Reviewer and Refactoring

The person in charge of reviewing my code was Harman Bains, who was tasked with reviewing Coin.java, Chest.java, Shield.java, and LosingByCoins.java. After reviewing, Harman was able to identify four code smells: three of them were in the Chest.java class and one was in LosingByCoins.java class.

Chest.java

1. **Dead Code**
   Harman identified that there was an unchecked suppressed warning applied for the sufficientCoins method, which is being accessed inside of the testing phase. As a result, Harman pointed out that this warning is unnecessary.

   Since the suppressed warning was unnecessary, I simply deleted the line that suppressed the "unused" warning. Testing was performed before and after to confirm that this refactoring caused no errors, and fixed the smell.

2. **Uncommunicative Name**
   The attemptOpening method had an unclear name, and didn't accurately describe to the user that the method describes a state, as opposed to describing an action. Since it returns a boolean, Harman pointed out that this method's name should inform the user that it intends to check or return a state of the chest.

   I refactored my code by changing the method name from attemptOpening to canOpenChest. Additionally, I refactored all test code that used this method to now use its new name. Testing was performed afterwards to ensure that no other methods still use the previous name, and all tests still pass.

3. **Poorly Structured Code**
   Harman noted that the attemptOpening method, now named canOpenChest after refactoring it, uses two seperate if statements for a boolean method. He pointed out that this code could be restructured to shorten the lines of code required for this method to perform the same function.

   Like Harman said, I was able to refactor this method by combining both if statements into one. Since this method returns a boolean, I simply checked all the conditions for when this method should return true inside the if statement, then made the method return false afterwards if the method still hasn't returned a boolean, indicating that the conditions in the if statement were not met.

LosingByCoins.java

1. **Redundant Comments**
   Harman pointed out that, although I did contribute a significant amount to the LosingByCoins.java class, my name was not added as an author in the JavaDoc comments for the class due to miscommunication. Additionally, Harman pointed out that some of the getters contained inline comments, or no comments at all, as opposed to JavaDoc comments, making them redundant.

   I refactored the code by adding my name to the authors section inside the JavaDoc comments for the class, and I also added JavaDoc comments for each method that didn't have one in this class.